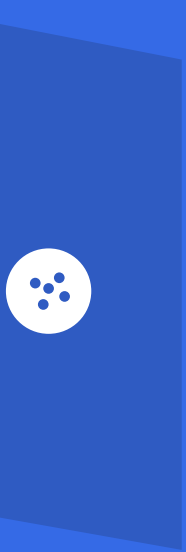
BASH SCRIPTING FUNDAMENTALS

Control Flow

Branching and looping with if, case, for, and while

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026



WHAT WE WILL COVER

Agenda

1. **Section 1: Exit Status Fundamentals** — 4 slides
2. **Section 2: Test Commands And Strings** — 4 slides
3. **Section 3: Patterns Regex And Integers** — 4 slides
4. **Section 4: File Tests** — 4 slides
5. **Section 5: Combining Conditions** — 4 slides
6. **Section 6: Case Statements** — 4 slides
7. **Section 7: For Loops** — 4 slides
8. **Section 8: While Until And Read** — 4 slides

9 **Section 9: Pipelines Select And Arrays** — 4 slides

10 **Section 10: Validation Patterns** — 4 slides

11 **Section 11: Nested Logic Retry And Timeouts** — 4 slides

12 **Section 12: Practice And Checkpoints** — 4 slides

1 Section 1

Exit Status Fundamentals

1.1 Exit status means success or failure · true is a successful command

- **Every** command finishes with a numeric exit status.
- Status 0 means success in Bash conditionals.
- Any nonzero status means failure for control flow.
- The **true** command exists only to return success.
- **It** is useful when a loop or branch needs a placeholder.
- It documents that success is intentional.

Examples

```
# Success status
true
echo $?
# Failure status
false
echo $?
# Always succeeds
if true; then echo yes; fi
while true; do break; done
```

1.2 false is a failing command · The dollar question variable

- The **false** command exists only to return failure.
- **It** is useful for examples and disabled branches.
- It behaves like any other failing command in if.
- $? stores the most recent command status.
- Read it immediately before another command changes it.
- Prefer direct if commands when possible.

Examples

```
# Always fails
if false; then echo yes; else echo no; fi
# Fallback
false || echo fallback
# Capture status
mkdir demo
status=$?
test -d demo
echo $?
```

1.3 Basic if then fi · Else branch

- An if statement runs a command as its condition.
- The then block runs only when the status is 0.
- The fi keyword closes the conditional.
- **else** handles the failed condition path.
- It runs when the if command returns nonzero.
- Use it for the single opposite outcome.

Examples

```
# Directory check
if test -d /tmp; then echo exists; fi
# Command check
if date >/dev/null; then echo ok; fi
# Path missing
if [[ -e app.log ]]; then echo found; else echo missing; fi
# Command failed
if ping -c1 host; then echo up; else echo down; fi
```

1.4 Elif chain

- **elif** tests another condition after a failure.
- **Branches** are checked from top to bottom.
- Only the first successful branch runs.

Examples

```
# Mode branch
if [[ $mode == dev ]]; then echo d; elif [[ $mode == prod ]]; then echo p; fi
# Number branch
if (( n < 0 )); then echo neg; elif (( n == 0 )); then echo zero; fi
```

2 Section 2

Test Commands And Strings

2.1 The test command · Single bracket test

- **test** evaluates file, string, and numeric expressions.
- It returns 0 for true and nonzero for false.
- **It** is a command, so if can run it directly.
- The [command is another spelling of **test**.
- It requires spaces after [and before].
- Quote variables to avoid word splitting.

Examples

```
# String test
if test -n "$name"; then echo set; fi
# File test
test -f config.env
echo $?
# Bracket form
if [ -f config.env ]; then echo ok; fi
[ "$x" = yes ] && echo yes
```

2.2 Double bracket test · Empty string with -z

- [[is a Bash conditional compound command.
- It avoids word splitting and pathname expansion.
- It supports pattern and regex matching.
- The -z operator tests for zero string length.
- **It** is true when a value is empty.
- Use it for required argument checks.

Examples

```
# Safer string test
if [[ $name == root ]]; then echo admin; fi
# Empty safe
[[ $value == "" ]] && echo empty
# Missing value
if [[ -z $user ]]; then echo missing; fi
# Argument check
[[ -z $1 ]] && echo need-arg
```

2.3 Nonempty string with -n · String equality

- The -n operator tests for nonzero **string** length.
- **It** is true when a value contains at least one character.
- **It** is clearer than comparing to an empty string.
- Use == inside [[for **string equality**.
- Use = inside [for portable test equality.
- Quote the right side when it must be literal text.

Examples

```
# Value present
if [[ -n $token ]]; then echo set; fi
# Prompt guard
[[ -n $answer ]] || echo blank
# Double bracket
[[ $env == prod ]] && echo live
# Single bracket
[ "$env" = prod ] && echo live
```

2.4 String inequality

- Use != to test that strings differ.
- In [[, the right side can act as a pattern when unquoted.
- Use quotes when a literal comparison matters.

Examples

```
# Not equal
if [[ $env != prod ]]; then echo safe; fi
# Literal compare
[[ $name != "*.tmp" ]] && echo literal
```

3 Section 3

Patterns Regex And Integers

3.1 Pattern matching with double brackets · Quoted patterns become literals

- Inside [[, == can match glob patterns.
- Leave the **pattern** unquoted to enable pattern matching.
- Quote the **pattern** when matching literal characters.
- A **quoted** right side of == is compared literally.
- This prevents glob characters from acting as patterns.
- Use this when user input may contain wildcard characters.

Examples

```
# Suffix pattern
[[ $file == *.txt ]] && echo text
# Prefix pattern
[[ $name == app-* ]] && echo app
# Literal star
[[ $value == "*.txt" ]] && echo literal
# Pattern star
[[ $value == *.txt ]] && echo match
```

3.2 Regex match operator · Integer equality operators

- The =~ operator matches a string against a regular expression.
- The **regex** should usually be unquoted in [[.
- Use anchors to require the whole value to match.
- Use -eq for **integer** equality in test expressions.
- Use -ne for **integer** inequality.
- These operators expect **integer** operands.

Examples

```
# Digits only
[[ $id =~ ^[0-9]+$ ]] && echo digits
# Name shape
[[ $name =~ ^[a-z_][a-z0-9_]*$ ]] && echo ok
# Equal
if [[ $count -eq 0 ]]; then echo empty; fi
# Not equal
[[ $retries -ne 3 ]] && echo keep-going
```

3.3 Integer less than operators · Integer greater than operators

- Use -lt to test less than.
- Use -le to test less than or equal.
- Prefer arithmetic syntax for complex arithmetic expressions.
- Use -gt to test greater than.
- Use -ge to test greater than or equal.
- Validate input before numeric comparisons.

Examples

```
# Less than
[[ $age -lt 18 ]] && echo minor
# At most
[[ $tries -le 5 ]] && echo allowed
# Greater than
[[ $size -gt 0 ]] && echo data
# At least
[[ $uid -ge 1000 ]] && echo user
```

3.4 Arithmetic conditions

- The (( )) form evaluates **arithmetic** expressions.
- A nonzero **arithmetic** result is true.
- **It** is concise for counters and loop logic.

Examples

```
# Counter positive
if (( count > 0 )); then echo yes; fi
# Even check
(( n % 2 == 0 )) && echo even
```

4 Section 4

File Tests

4.1 Path exists with -e · Regular file with -f

- The -e test checks whether a **path** exists.
- It does not care what type of **path** it is.
- Use it before choosing a more specific file test.
- The -f test checks for a **regular** file.
- **It** is false for directories and most special files.
- Use it before reading expected file content.

Examples

```
# Any path
[[ -e config.env ]] && echo exists
# Missing path
[[ ! -e cache ]] && echo absent
# Config file
if [[ -f config.env ]]; then source config.env; fi
# Skip nonfile
[[ -f $path ]] || echo not-file
```

4.2 Directory with -d · Symbolic link with -L

- The -d test checks for a **directory**.
- **It** is useful before listing or writing inside a path.
- It distinguishes directories from regular files.
- The -L test checks whether a path is a **symbolic** link.
- **It** is useful before replacing or resolving links.
- It tests the link itself, not only the target.

Examples

```
# Directory exists
[[ -d logs ]] && echo has-logs
# Create if missing
[[ -d tmp ]] || mkdir tmp
# Link check
[[ -L current ]] && echo symlink
# Remove link
if [[ -L old ]]; then rm old; fi
```

4.3 Readable path with -r · Writable path with -w

- The -r test checks read permission for the current user.
- It helps produce clearer errors before opening files.
- Permission can still change after the test.
- The -w test checks write permission for the current user.
- **It** is useful before writing files or directories.
- A **writable** directory allows creating entries inside it.

Examples

```
# Readable file
[[ -r config.env ]] && echo can-read
# Guard read
if [[ ! -r $file ]]; then echo unreadable; fi
# Writable file
[[ -w report.txt ]] && echo can-write
# Writable directory
[[ -w logs ]] || echo no-write
```

4.4 Executable path with -x

- The -x test checks execute permission.
- For files, it means the file can be executed.
- For directories, it means the directory can be searched.

Examples

```
# Script executable
[[ -x ./deploy ]] && ./deploy
# Directory access
[[ -x /tmp ]] && echo enterable
```

5 Section 5

Combining Conditions

5.1 Nonempty file with -s · And with double ampersand

- The -s test checks that a file exists and has size greater than zero.
- **It** is useful before processing generated output.
- It avoids treating empty files as useful data.
- The && operator runs the right command only after success.
- Inside [[, && combines true conditions.
- Use it for requirements that must all pass.

Examples

```
# Has content
[[ -s results.txt ]] && echo ready
# Empty report
[[ -s report.txt ]] || echo empty
# Two tests
[[ -f $file && -r $file ]] && echo usable
# Command chain
mkdir out && echo made
```

5.2 Or with double pipe · Negation with exclamation

- The || operator runs the right command only after failure.
- Inside [[, || accepts either condition.
- Use it for alternatives **or** fallbacks.
- The ! operator reverses success and failure.
- It can negate commands **or** conditions.
- Use it sparingly so branches remain readable.

Examples

```
# Either suffix
[[ $f == *.jpg || $f == *.png ]] && echo image
# Fallback
cd app || exit 1
# Not a directory
if [[ ! -d logs ]]; then mkdir logs; fi
# Command not found
if ! command -v jq; then echo install-jq; fi
```

5.3 Grouping inside double brackets · Short circuit mkdir fallback

- Parentheses group conditions inside [[.
- **Grouping** makes precedence explicit.
- **Spaces** are required around grouped expressions.
- mkdir returns failure when it cannot create a directory.
- A || branch can stop the script after mkdir fails.
- Use -p when parent creation and existing directories are acceptable.

Examples

```
# Group alternatives
[[ -f $p && ( $p == *.sh || $p == *.bash ) ]] && echo shell
# Group modes
[[ $env == prod && ( $role == web || $role == api ) ]] && echo deploy
# Stop on failure
mkdir -p logs || exit 1
# Message then stop
mkdir cache || { echo failed; exit 1; }
```

5.4 Spacing mistakes in tests

- The [command requires separate words for each token.
- Missing spaces change the command name or arguments.
- Double brackets also need spaces around the outer tokens.

Examples

```
# Correct bracket
[ -n "$name" ] && echo set
# Correct double bracket
[[ -n $name ]] && echo set
```

6 Section 6

Case Statements

6.1 Case esac shape · Exact case patterns

- **case** compares one word against ordered patterns.
- Each matching arm runs its commands.
- The esac keyword closes the statement.
- A plain word pattern matches that **exact** value.
- **Patterns** are checked in order.
- Use **exact** patterns for command names and modes.

Examples

```
# Basic case
case $cmd in start) echo go ;; stop) echo halt ;; esac
# With variable
case $env in prod) echo live ;; *) echo other ;; esac
# Mode
case $mode in debug) echo debug ;; esac
# Answer
case $answer in yes) echo ok ;; no) echo skip ;; esac
```

6.2 Case alternatives with pipe · Case glob patterns

- The | character separates patterns in one **case** arm.
- The same commands run for any listed alternative.
- Use it to handle aliases cleanly.
- **case** patterns use shell glob syntax.
- A star can match any string segment.
- Use globs for suffixes, prefixes, and ranges.

Examples

```
# Yes answers
case $ans in y|yes) echo yes ;; n|no) echo no ;; esac
# Start aliases
case $cmd in run|start) echo starting ;; esac
# Suffix
case $file in *.sh) echo shell ;; esac
# Range
case $key in [0-9]) echo digit ;; esac
```

6.3 Default case arm · Case arm terminator

- The * pattern is commonly used as a **default**.
- Place it last so specific patterns are checked first.
- Use it for errors or fallback behavior.
- The ;; token ends a **case** arm.
- Bash does not fall through by **default**.
- For bootcamp scripts, prefer explicit separate arms.

Examples

```
# Unknown command
case $cmd in start) echo go ;; *) echo unknown ;; esac
# Default value
case $color in red) echo stop ;; *) echo pass ;; esac
# Two arms
case $n in 1) echo one ;; 2) echo two ;; esac
# Default stop
case $x in a) echo A ;; *) echo other ;; esac
```

6.4 Menus with case

- case pairs naturally with menu choices.
- Each arm can call one action.
- The default arm handles invalid input.

Examples

```
# Choice
case $choice in 1) echo add ;; 2) echo remove ;; *) echo bad ;; esac
# Letter menu
case $op in q) exit 0 ;; h) echo help ;; esac
```

7 Section 7

For Loops

7.1 For in word list · For loop over quoted values

- A for loop assigns one word at a time to a variable.
- The do block runs once for each word.
- The done keyword closes the loop.
- Quoted items stay single loop values.
- Unquoted expansions can split into multiple words.
- **Arrays** are the cleanest way to keep values intact.

Examples

```
# Names
for name in ann bob; do echo $name; done
# Modes
for mode in dev test prod; do echo $mode; done
# Quoted words
for item in "two words" three; do echo "$item"; done
items=("two words" three)
for item in "${items[@]}"; do echo "$item"; done
```

7.2 For loop over arguments · For loop over globs

- A for loop without in uses the script arguments.
- Inside functions, it uses the function arguments.
- Use this when every argument gets the same handling.
- Globs expand to matching pathnames before the loop starts.
- Quote the loop variable inside the body.
- Use this for simple file batches.

Examples

```
# Implicit args
for arg; do echo "$arg"; done
# Explicit args
for arg in "$@"; do echo "$arg"; done
# Shell scripts
for f in *.sh; do echo "$f"; done
# Logs
for log in logs/*.log; do gzip "$log"; done
```

7.3 Empty glob care · C style for loop

- An unmatched glob remains literal by default.
- That can make a loop process a fake filename.
- Check existence inside the loop or enable nullglob deliberately.
- The for ((init; test; update)) form uses arithmetic.
- **It** is useful for numeric counters.
- The loop continues while the test expression is nonzero.

Examples

```
# Guard glob
for f in *.csv; do [[ -e $f ]] || continue; echo "$f"; done
# Use nullglob
shopt -s nullglob
for f in *.csv; do echo "$f"; done
# Count up
for ((i=0; i<3; i++)); do echo $i; done
for ((i=3; i>0; i--)); do echo $i; done
```

7.4 Arithmetic step in for

- The update expression can change by more than one.
- Use += and -= for clear step sizes.
- Keep loop bounds easy to audit.

Examples

```
# Step by two
for ((i=0; i<10; i+=2)); do echo $i; done
# Halve
for ((n=8; n>0; n/=2)); do echo $n; done
```

8 Section 8

While Until And Read

8.1 While loop · Until loop

- **while** repeats while its condition command succeeds.
- The **condition** is checked before each iteration.
- Use it when the number of iterations is not fixed.
- until repeats **while** its condition command fails.
- It stops when the condition succeeds.
- Use it when waiting for success reads naturally.

Examples

```
# Counter
i=0
while (( i < 3 )); do echo $((i++)); done
# Readiness
while [[ ! -f done.flag ]]; do sleep 1; done
# Wait for file
until [[ -f ready ]]; do sleep 1; done
until curl -fsS http://localhost; do sleep 2; done
```

8.2 Break exits a loop · Continue skips to the next iteration

- **break** stops the nearest enclosing loop.
- **It** is useful when a success condition appears early.
- Avoid hidden **break** paths in long loop bodies.
- **continue skips** the rest of the current loop body.
- **It** is useful for filtering invalid items.
- Use early **continue** statements to reduce nesting.

Examples

```
# Find first
for f in *.log; do [[ -s $f ]] && { echo $f; break; }; done
# Stop menu
while true; do read -r x; [[ $x == q ]] && break; done
# Skip dirs
for p in *; do [[ -d $p ]] && continue; echo $p; done
# Skip blanks
while read -r line; do [[ -z $line ]] && continue; echo $line; done
```

8.3 Infinite loops need exits · Reading lines with while read

- **Infinite** loops are intentional only with a clear exit path.
- Use break, return, or exit for controlled termination.
- Add sleeps when polling to avoid busy waiting.
- read gets one line of input at a time.
- A while read loop is standard for line processing.
- Redirect the file into the loop for predictable scope.

Examples

```
# Menu loop
while true; do read -r c; [[ $c == q ]] && break; done
# Polling loop
while true; do [[ -f ready ]] && break; sleep 1; done
# Read file
while read -r line; do echo "$line"; done < names.txt
n=0
while read -r line; do echo $((++n)): $line; done < file
```

8.4 IFS read r

- **IFS**= preserves leading and trailing whitespace for read.
- The -r option prevents backslash escapes.
- Together they are the safe default for raw lines.

Examples

```
# Safe read
while IFS= read -r line; do printf '%s\n' "$line"; done < file
# Prompt read
IFS= read -r answer
```

9 Section 9

Pipelines Select And Arrays

9.1 Pipelines in if · Pipeline while read scope

- An if condition can be a **pipeline**.
- The **pipeline** status normally comes from the last command.
- pipefail changes how **pipeline** failures are reported.
- A **pipeline** may run the while loop in a subshell.
- Variable changes can disappear after the loop.
- Use redirection when the final variable value matters.

Examples

```
# Search output
if ps aux | grep -q '[s]shd'; then echo running; fi
# Count match
if printf '%s\n' a b | grep -q b; then echo found; fi
# Pipeline count
printf '%s\n' a b | while read -r x; do echo $x; done
count=0
printf x | while read -r _; do ((count++)); done
```

9.2 Redirected while read · Mapfile intro

- Redirecting input into a loop keeps the loop in the current shell.
- **This** is preferred when updating variables.
- It also makes the input source easy to spot.
- **mapfile** reads lines into an array.
- **It** is useful when later code needs random access.
- Use -t to remove trailing newlines.

Examples

```
# Count lines
count=0
while read -r _; do ((count++)); done < file
# Collect names
while read -r name; do echo "hi $name"; done < names.txt
# Read array
mapfile -t lines < names.txt
echo "${#lines[@]}"
mapfile -t rows < data.txt
echo "${rows[0]}"
```

9.3 Readarray alias · Select menu

- **readarray** is another name for mapfile.
- **Both** are Bash builtins in Bash 4 and later.
- Use the spelling your team finds clearer.
- **select** builds a numbered menu from words.
- It stores the selected item in the loop variable.
- Use break when the user makes a valid choice.

Examples

```
# Readarray
readarray -t files < list.txt
printf '%s\n' "${files[@]}"
# Count rows
readarray -t rows < data.txt
echo "${#rows[@]}"
# Simple select
select env in dev prod quit; do echo "$env"; break; done
select op in add quit; do case $op in quit) break ;; esac; done
```

9.4 PS3 select prompt

- **PS3** controls the prompt displayed by select.
- Set it before the select loop.
- Keep prompts short and action oriented.

Examples

```
# Custom prompt
PS3='Choose: '
select item in one two; do echo "$item"; break; done
# Menu prompt
PS3='Action: '
select op in build test; do break; done
```

10

Section 10

Validation Patterns

10.1 Validate integers with regex · Validate nonnegative counts

- Use =~ to confirm input is an integer before math.
- Anchors prevent partial matches.
- A sign can be allowed explicitly when needed.
- Counts often require zero or positive values.
- Regex validation avoids arithmetic errors from text.
- Follow validation with numeric range checks if needed.

Examples

```
# Unsigned integer
[[ $n =~ ^[0-9]+$ ]] || echo not-int
# Signed integer
[[ $n =~ ^-?[0-9]+$ ]] && echo int
# Count value
if [[ $count =~ ^[0-9]+$ ]]; then echo ok; fi
# Positive value
[[ $n =~ ^[0-9]+$ && $n -gt 0 ]] && echo positive
```

10.2 Validate path existence · Validate readable files

- Check that a path exists before acting on it.
- Use -e when any path type is acceptable.
- Report the path in error messages.
- Readable file checks combine -f and -r.
- This rejects directories and unreadable files.
- Use it before parsing configuration.

Examples

```
# Require path
[[ -e $path ]] || { echo "missing: $path"; exit 1; }
# Optional path
if [[ -e $path ]]; then echo found; fi
# Config guard
[[ -f $cfg && -r $cfg ]] || exit 1
# Read branch
if [[ -r $file ]]; then wc -l < "$file"; fi
```

10.3 Validate writable directories · Validate commands with command v

- Writable directory checks combine -d and -w.
- Use them before creating output files.
- This gives faster feedback than a failed write later.
- command -v checks whether a command can be found.
- It works for builtins, functions, and executables.
- Use it before relying on optional tools.

Examples

```
# Output directory
[[ -d $out && -w $out ]] || exit 1
# Create output
[[ -d out ]] || mkdir out
# Need jq
if ! command -v jq >/dev/null; then echo need-jq; fi
# Need git
command -v git >/dev/null || exit 1
```

10.4 Error branches

- Validation failures should take a clear branch.
- Use exit or return after fatal errors.
- Keep the success path easy to read.

Examples

```
# Fatal branch
[[ -n $name ]] || { echo name-required; exit 1; }
# Nonfatal branch
if [[ -z $tag ]]; then tag=latest; fi
```

11 Section 11

Nested Logic Retry And Timeouts

11.1 Nested if · Guard clauses reduce nesting

- **Nested if** statements express dependent decisions.
- Each inner test should rely on the outer branch being true.
- Too much nesting can hide the main path.
- A **guard** clause exits early when a requirement fails.
- It keeps the main branch less indented.
- Use it for fatal validation checks.

Examples

```
# Nested check
if [[ -d repo ]]; then if [[ -f repo/config ]]; then echo ok; fi; fi
# Nested command
if cd repo; then if git status; then echo clean; fi; fi
# Require arg
[[ -n $1 ]] || exit 2
echo "arg=$1"
[[ -d src ]] || exit 1
echo build
```

11.2 Retry loop · Sleep between retries

- A **retry loop** repeats a command after failure.
- Limit retries so failures do not run forever.
- Break as soon as the command succeeds.
- **sleep** adds delay between attempts.
- It protects services from tight retry loops.
- Keep **retry** delays visible in the loop body.

Examples

```
# Three tries
for i in 1 2 3; do curl -fsS URL && break; done
# Retry flag
ok=0
for i in 1 2 3; do command && { ok=1; break; }; done
# Sleep retry
for i in 1 2 3; do curl -fsS URL && break; sleep 2; done
until nc -z host 80; do sleep 1; done
```

11.3 Timeout command · Failure limit

- **timeout** limits how long a command may run.
- It returns **failure** when the time limit is reached.
- Use it around network or interactive commands.
- Track attempts when retrying manually.
- Stop after a known maximum **failure** count.
- Return or exit after exhausting retries.

Examples

```
# Limit curl
if timeout 5 curl -fsS URL; then echo ok; fi
# Limit read
if timeout 10 bash -c 'read -r x'; then echo got; fi
# Attempt counter
tries=0
until cmd; do ((++tries == 3)) && exit 1; done
for ((i=1; i<=3; i++)); do cmd && break; done
```

11.4 Break on success

- Successful work often ends a search loop.
- Use **break** immediately after the success action.
- This avoids processing unnecessary later items.

Examples

```
# First match
for f in *.txt; do grep -q needle "$f" && { echo "$f"; break; }; done
# First port
for p in 8000 8001; do nc -z localhost $p && { echo $p; break; }; done
```

12 Section 12

Practice And Checkpoints

12.1 If command practice · Case dispatch practice

- Use **if** directly with the command being tested.
- Avoid checking $? when an **if** command is clearer.
- Redirect noisy command output when only status matters.
- Use **case** to dispatch small command names.
- Keep each arm focused on one action.
- Route unknown commands to a usage error.

Examples

```
# Grep status
if grep -q root /etc/passwd; then echo found; fi
# Git status
if git rev-parse --is-inside-work-tree >/dev/null 2>&1; then echo repo; fi
# Dispatch
case $1 in build) echo build ;; test) echo test ;; *) echo usage ;; esac
# Aliases
case $1 in rm|delete) echo remove ;; *) echo unknown ;; esac
```

12.2 File loop practice · Prompt until valid

- Combine globs with **file** tests for safe batches.
- Skip nonmatching or invalid paths early.
- Quote each pathname inside the loop body.
- A while loop can repeat until input passes validation.
- Validate immediately after each read.
- Use break when a valid answer is accepted.

Examples

```
# Readable logs
for f in logs/*.log; do [[ -r $f ]] || continue; echo "$f"; done
# Executable scripts
for f in bin/*; do [[ -x $f ]] && echo "$f"; done
# Ask number
while true; do read -r n; [[ $n =~ ^[0-9]+$ ]] && break; done
# Ask yes no
while read -r a; do [[ $a == y || $a == n ]] && break; done
```

12.3 Select and case practice · Missing fi mistake

- **select** provides the menu display.
- case handles the selected action.
- Invalid or empty choices should continue the menu.
- Every if statement must end with fi.
- **Missing** fi often reports a syntax error later in the file.
- Indentation helps humans see the matching structure.

Examples

```
# Action menu
select op in build quit; do case $op in quit) break ;; build) echo build ;; esac; done
# Environment menu
select env in dev prod; do [[ -n $env ]] && break; done
# Closed if
if [[ -n $x ]]; then echo set; fi
# Nested closed
if [[ -d a ]]; then if [[ -f a/b ]]; then echo ok; fi; fi
```

12.4 Control flow checkpoint

- Choose if for command status and simple decisions.
- Choose case for one value matched against many patterns.
- Choose loops based on whether input is a list, condition, or menu.

Examples

```
# Decision
[[ -f config ]] && echo file || echo missing
# Loop
for f in *.sh; do bash -n "$f"; done
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
03 · Control Flow